International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Lab 8 – Function Overriding (Group B2)

Date: 27.10.2026 **Semester:** B.Tech 3rd Semester **Group:** B2

Instructions

- Write and execute the C++ program for each problem.
- Use inheritance and function overriding wherever required.
- Match the function signatures in the base and derived classes.
- Display clear and meaningful output.
- Use virtual functions and base-class pointers or references wherever specified.
- Save all programs in your assigned GitHub repository.
- The repository submission must be completed by 12:00 AM on the same day to receive marks.
- Submit the lab record in the prescribed format.

1. **Game Character Battle**

   Create a base class `Character` with a function `attack()`.

   Derive classes `Warrior` and `Archer`. Override `attack()` so that a warrior deals damage equal to $20 + 2\times$ strength, while an archer deals damage equal to $15 + 3\times$ agility.

   Display the damage dealt by each character.

2. **Smart Parking Fee Calculator**

   Create a base class `ParkingSlot` with a function `calculateFee(int hours)`.

   Derive classes `TwoWheelerSlot` and `FourWheelerSlot`. Override the function to calculate parking fees at rates of Rs. 10 and Rs. 30 per hour, respectively.

   If the vehicle stays for more than 5 hours, apply a 10% discount to the total fee. Display the final fee for each vehicle type.

3. **File Compression Simulator**

   Create a base class `Compressor` with a function `compress(double fileSize)`.

   Derive classes `TextCompressor` and `ImageCompressor`. Override the function to calculate the compressed file size, assuming text files retain 40% of their original size and image files retain 70%.

   Display the original and compressed file sizes.

4. **Food Delivery Time Estimator**

   Create a base class `Delivery` with a function `estimateTime(double distance)`.

   Derive classes `BikeDelivery` and `DroneDelivery`. Override the function to estimate delivery time using speeds of 20 km/h and 40 km/h, respectively.

   Assume the bike requires an additional 5 minutes for traffic, while the drone requires an additional 2 minutes for take-off and landing. Display the estimated time in minutes.

5. **Smart Irrigation Controller**

   Create a base class `IrrigationSystem` with a function `calculateWater(double moisture)`.

   Derive classes `DripIrrigation` and `SprinklerIrrigation`. Override the function so that drip irrigation supplies $2 \times (100 - \textit{moisture})$ litres, while sprinkler irrigation supplies $3 \times (100 - \textit{moisture})$ litres.

   Assume soil moisture is given as a percentage between 0 and 100. Display the required water for each system.

6. **Cybersecurity Threat Scanner**

   Create a base class `SecurityScanner` with a function `scan(int threats)`.

   Derive classes `BasicScanner` and `AdvancedScanner`. Override the function so that the basic scanner reports the number of threats detected, while the advanced scanner reports the number detected and the number remaining after removing 90% of the detected threats.

   Display the results for both scanners.

7. **Cricket Target Calculator**

   Create a base class `MatchFormat` with a function `calculateTarget(int firstInningsScore)`.

   Derive classes `T20Match` and `ODIMatch`. Override the function to calculate a target score by adding a required run margin of 1% of the first-innings score for T20 matches and 2% for ODI matches. Round the margin up to the next integer when necessary.

   Display the target score for each format.

8. **Cloud Resource Cost Calculator**

   Create a base class `CloudResource` with a function `calculateCost(double cpuHours)`.

   Derive classes `StandardInstance` and `PremiumInstance`. Override the function to calculate costs at Rs. 5 and Rs. 12 per CPU-hour, respectively.

   If CPU usage exceeds 100 hours, apply a 15% discount to the total cost. Display the final cost for each instance.

9. **Robot Navigation System**

   Create a base class `Robot` with a virtual function `move(double distance)`.

   Derive classes `WheeledRobot` and `TrackedRobot`. Override `move()` to calculate travel time using speeds of 2 m/s and 1.2 m/s, respectively.

   Create base-class pointers to the derived objects and call `move()` through these pointers to demonstrate runtime polymorphism.

10. AI Recommendation Engine

Create a base class `Recommender` with a virtual function `generateRecommendation(int userRating, int popularity)`.

Derive classes `PersonalizedRecommender` and `TrendingRecommender`.

Override the function in `PersonalizedRecommender` to calculate a score as

$$0.7 \times \text{userRating} + 0.3 \times \text{popularity}.$$

Override the function in `TrendingRecommender` to calculate a score as

$$0.3 \times \text{userRating} + 0.7 \times \text{popularity}.$$

Assume both inputs are scores between 0 and 100. Create a base-class pointer to each derived object and invoke the function to display the recommendation score, demonstrating runtime polymorphism.